

Chrome DevTools Implementation Report

HW10 - CS484 Web Programming

Project: Discord Kitten DevTools demo copy

Prepared for the Chrome DevTools console logging and JavaScript debugging assignment

This report documents the practical Chrome DevTools examples added to the Project 2 copy. The app now includes console logging demos, browser-error reproducers, message filtering examples, breakpoint-friendly code paths, and a safe fix flow after reproducing the bug.

Implemented Areas

- Message logging with `console.info`, `console.warn`, `console.error`, `console.table`, `console.group`, and styled console output.
- Browser message examples for a 404 fetch error, a `TypeError`, and a long-running operation.
- Filtering examples for text, regular expressions, message source tags, and user-generated messages.
- Debugging examples for breakpoints, variable inspection, watch expressions, pause on exception, step-through practice, and a fixed version of the bug.

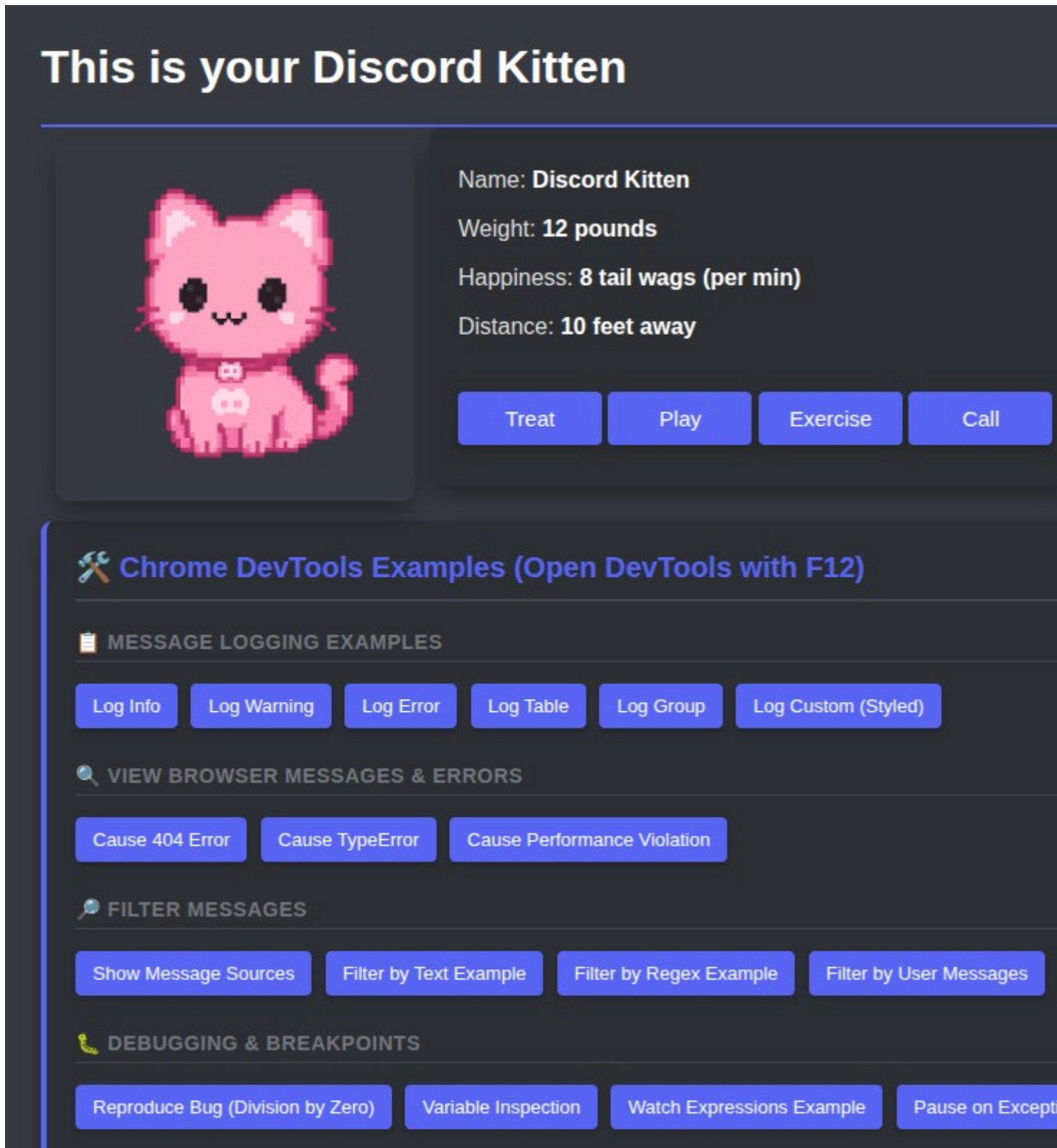
Key Files

- `index.html` adds the DevTools demo buttons.
- `script.ts` contains the new console and debugging examples.
- `script.js` is the compiled bundle used by the browser.

- [assets/DEVTOOLS_GUIDE.html](#) and the screenshots folder support the written report.

Figure 1. Main App Overview

The copied Project 2 page now exposes the DevTools demo buttons directly in the UI. This is the launch point for the logging, filtering, and debugging workflows described in the assignment.



The app now includes all of the HW10 demo controls in one place.

Figure 2. Filtering Guide

The guide section explains how to filter console messages by text, regular expression, source tag, and user-message type. This matches the new console output added in `script.ts`.

-
-

Filtering behavior is tied to the source-tagged and bracketed console messages in the demo code.

Figure 3. Debugging and Breakpoints Guide

The debugging section covers line breakpoints, variable inspection, watch expressions, pause on exception, and stepping through code. The app includes dedicated functions for each of those flows, including a debugger-statement pause and a fixed division path.

-
-

The debugging code paths are designed to be easy to pause and inspect in Chrome Sources.

Figure 4. Logging Workflow Reference

This guide view shows the logging area that corresponds to the console info, warning, error, table, group, and custom-style examples in the app.



HW10: Chrome DevTools Imple

Comprehensive guide to all Chrome DevTools features implemented in the Discord Kitten p



Message Logging Examples

The Console panel in DevTools displays messages logged by your application. Different log

1.1 Log Info

Used for informational messages about the application's state and operations.

```
console.info("📘 [INFO] Pet was just fed! Happiness is now: 9");
```

In DevTools: Shows as a regular message with an info icon (📘)

- ✓ Appears in Console tab
- ✓ Blue info icon next to message
- ✓ Use for non-critical application events

1.2 Log Warning

Used for potential issues or warnings that shouldn't occur during normal operation.

```
console.warn("⚠️ [WARNING] Pet happiness is dropping: 5");
```

This section maps to the message logging controls implemented in the page.

Figure 5. Console Logging Reference

The next guide section reinforces the browser logging flows and how they appear in the Console tab when the demo buttons are clicked.

1.1 Log Info

Used for informational messages about the application's state and operations.

```
console.info("📘 [INFO] Pet was just fed! Happiness is now: 9");
```

In DevTools: Shows as a regular message with an info icon (📘)

- ✓ Appears in Console tab
- ✓ Blue info icon next to message
- ✓ Use for non-critical application events

1.2 Log Warning

Used for potential issues or warnings that shouldn't occur during normal operation.

```
console.warn("⚠️ [WARNING] Pet happiness is dropping: 5");
```

In DevTools: Shows as a warning with yellow background and warning icon (⚠️)

- ✓ Yellow warning background in console
- ✓ Visible in Console and Issues tab
- ✓ Use for conditions that should be investigated

1.3 Log Error

Used for actual errors in the application.

```
console.error("❌ [ERROR] Pet weight is critically low: 0");
```

In DevTools: Shows with red background and error icon (❌)

Use this view while collecting the screenshots for the six message logging examples.

Figure 6. Network Error Reference

This guide section describes the 404 network request scenario and the browser-generated message flow that appears after triggering the missing image fetch.

Used for potential issues or warnings that shouldn't occur during normal operation.

```
console.warn("⚠ [WARNING] Pet happiness is dropping: 5");
```

In DevTools: Shows as a warning with yellow background and warning icon (⚠)

- ✓ Yellow warning background in console
- ✓ Visible in Console and Issues tab
- ✓ Use for conditions that should be investigated

1.3 Log Error

Used for actual errors in the application.

```
console.error("❌ [ERROR] Pet weight is critically low: 0");
```

In DevTools: Shows with red background and error icon (❌)

- ✓ Red error background in console
- ✓ Increments error counter at top of console
- ✓ Use for exceptional conditions

1.4 Log Table

Display objects and arrays as formatted tables in the console.

```
console.table(petInfo);  
console.table(petHistory);
```

In DevTools: Renders data as an HTML table

- ✓ Organized columns for object properties

This aligns with the browser error examples in the assignment guide.

Figure 7. TypeError Reference

The TypeError workflow is documented here so the console stack trace can be compared with the app's intentional invalid-property access example.

```
console.error("❌ [ERROR] Pet weight is critically low: 0");
```

In DevTools: Shows with red background and error icon (❌)

- ✓ Red error background in console
- ✓ Increments error counter at top of console
- ✓ Use for exceptional conditions

1.4 Log Table

Display objects and arrays as formatted tables in the console.

```
console.table(petInfo);  
console.table(petHistory);
```

In DevTools: Renders data as an HTML table

- ✓ Organized columns for object properties
- ✓ Sortable by clicking column headers
- ✓ Great for viewing structured data

1.5 Log Group

Organize console messages into hierarchical groups that can be collapsed/expanded.

```
console.group("🐱 Pet Interaction Group");  
console.log("User clicked Treat button");  
console.log("Happiness updated");  
console.groupEnd();
```

In DevTools: Messages appear nested with collapsible arrow

- ✓ Creates collapsible section in console

This section supports the browser-message debugging example in the app.

Figure 8. Performance Violation Reference

The performance section explains the long-running operation demo and the record/stop workflow in the Performance panel.

In DevTools: Renders data as an HTML table

- ✓ Organized columns for object properties
- ✓ Sortable by clicking column headers
- ✓ Great for viewing structured data

1.5 Log Group

Organize console messages into hierarchical groups that can be collapsed/expanded.

```
console.group("🐱 Pet Interaction Group");  
console.log("User clicked Treat button");  
console.log("Happiness updated");  
console.groupEnd();
```

In DevTools: Messages appear nested with collapsible arrow

- ✓ Creates collapsible section in console
- ✓ Supports nested groups (console.group within group)
- ✓ Improves console readability for complex operations

1.6 Log Custom

Style console messages using CSS to make them stand out.

```
const style = "color: #ff6b9d; font-size: 16px; font-weight: bold";  
console.log("%c💥 Custom Styled Message!", style);
```

In DevTools: Appears with custom styling (colors, sizes, fonts)

- ✓ Use %c placeholder for styled section
- ✓ CSS style properties can be applied

This corresponds to the heavy computation example in the project code.

Figure 9. Text Filtering Reference

This view shows the filtering guidance that matches the app's source-tagged and text-filterable console messages.

```
console.log(groupName, //
```

In DevTools: Messages appear nested with collapsible arrow

- ✓ Creates collapsible section in console
- ✓ Supports nested groups (console.group within group)
- ✓ Improves console readability for complex operations

1.6 Log Custom

Style console messages using CSS to make them stand out.

```
const style = "color: #ff6b9d; font-size: 16px; font-weight: bold;";
console.log("%c💥 Custom Styled Message!", style);
```

In DevTools: Appears with custom styling (colors, sizes, fonts)

- ✓ Use %c placeholder for styled section
- ✓ CSS style properties can be applied
- ✓ Multiple %c styles in one message



View Messages Logged by the Browser

Chrome automatically logs errors and issues that occur in your application:

2.1 Cause 404 Network Error

When you request a resource that doesn't exist on the server.

Use this section while explaining the filter-by-text and source-tag examples.

Figure 10. Regex Filtering Reference

The regex guidance lines up with the bracketed messages emitted by the new filter-by-regex demo in the app.

In DevTools: Appears with custom styling (colors, sizes, fonts)

- ✓ Use %c placeholder for styled section
- ✓ CSS style properties can be applied
- ✓ Multiple %c styles in one message



View Messages Logged by the Browser

Chrome automatically logs errors and issues that occur in your application:

2.1 Cause 404 Network Error

When you request a resource that doesn't exist on the server.

Step 1: Click "Cause 404 Error" button on the main page

Step 2: Open DevTools (F12)

Step 3: Go to Console tab

Step 4: Look for "❌ Network Error - 404"

Step 5: Go to Network tab to see the failed request in red

What to look for: Network tab shows GET request with 404 status code in red text

2.2 Cause TypeError

This section supports the regular-expression filter example.

Figure 11. Message Source Reference

This screenshot anchors the source-tag filtering section, which is why the app prints messages like [PET-CONTROLLER] and [UI-RENDERER].

2.1 Cause 404 Network Error

When you request a resource that doesn't exist on the server.

Step 1: Click "Cause 404 Error" button on the main page

Step 2: Open DevTools (F12)

Step 3: Go to Console tab

Step 4: Look for "❌ Network Error - 404"

Step 5: Go to Network tab to see the failed request in red

What to look for: Network tab shows GET request with 404 status code in red text

2.2 Cause TypeError

When code attempts an invalid operation on a variable.

Step 1: Click "Cause TypeError" button

Step 2: In DevTools Console, see "🔴 Deliberately causing a TypeError..."

Step 3: Look for error message about calling method on undefined

Step 4: Stack trace shows which line caused the error

What to look for: Red error message with file name and line number in stack trace

The source tags are intended to be easy to isolate in the Console.

Figure 12. Breakpoint Setup Reference

This guide view is used to explain how to set a line-of-code breakpoint before triggering the variable-inspection and pause examples.

What to look for: Network tab shows GET request with 404 status code in red text

2.2 Cause TypeError

When code attempts an invalid operation on a variable.

Step 1: Click "Cause TypeError" button

Step 2: In DevTools Console, see "🔴 Deliberately causing a TypeError..."

Step 3: Look for error message about calling method on undefined

Step 4: Stack trace shows which line caused the error

What to look for: Red error message with file name and line number in stack trace

2.3 Cause Performance Violation

Long-running operations that block the main thread and cause performance issues.

Step 1: Click "Cause Performance Violation" button

Step 2: Go to Performance tab in DevTools

Step 3: Click the circle to record, then click button again

Step 4: Stop recording and view the performance timeline

Step 5: Look for a long continuous yellow/orange bar indicating heavy CPU usage

This ties directly to the Sources-panel breakpoint workflow.

Figure 13. Scope Pane Reference

The scope-pane instructions explain where local variables appear while paused in Sources, matching the app's inspection-friendly function bodies.

Step 4: Stack trace shows which line caused the error

What to look for: Red error message with file name and line number in stack trace

2.3 Cause Performance Violation

Long-running operations that block the main thread and cause performance issues.

Step 1: Click "Cause Performance Violation" button

Step 2: Go to Performance tab in DevTools

Step 3: Click the circle to record, then click button again

Step 4: Stop recording and view the performance timeline

Step 5: Look for a long continuous yellow/orange bar indicating heavy CPU usage

What to look for: Long task indicator in the Performance timeline

Filter Messages

Chrome DevTools provides powerful filtering capabilities in the Console:

3.1 Filter by Log Level

Show only messages of a specific severity level.

This section supports the variable inspection examples.

Figure 14. Watch Expressions Reference

This guide screenshot explains the watch-expression panel and how to track values while stepping through the debugging examples.

Step 4: Stop recording and view the performance timeline

Step 5: Look for a long continuous yellow/orange bar indicating heavy CPU usage

What to look for: Long task indicator in the Performance timeline

Filter Messages

Chrome DevTools provides powerful filtering capabilities in the Console:

3.1 Filter by Log Level

Show only messages of a specific severity level.

In Console tab: Click the filter funnel icon

Uncheck: "Errors", "Warnings", or "Logs" as needed

Result: Console shows only messages matching your filter

3.2 Filter by Text

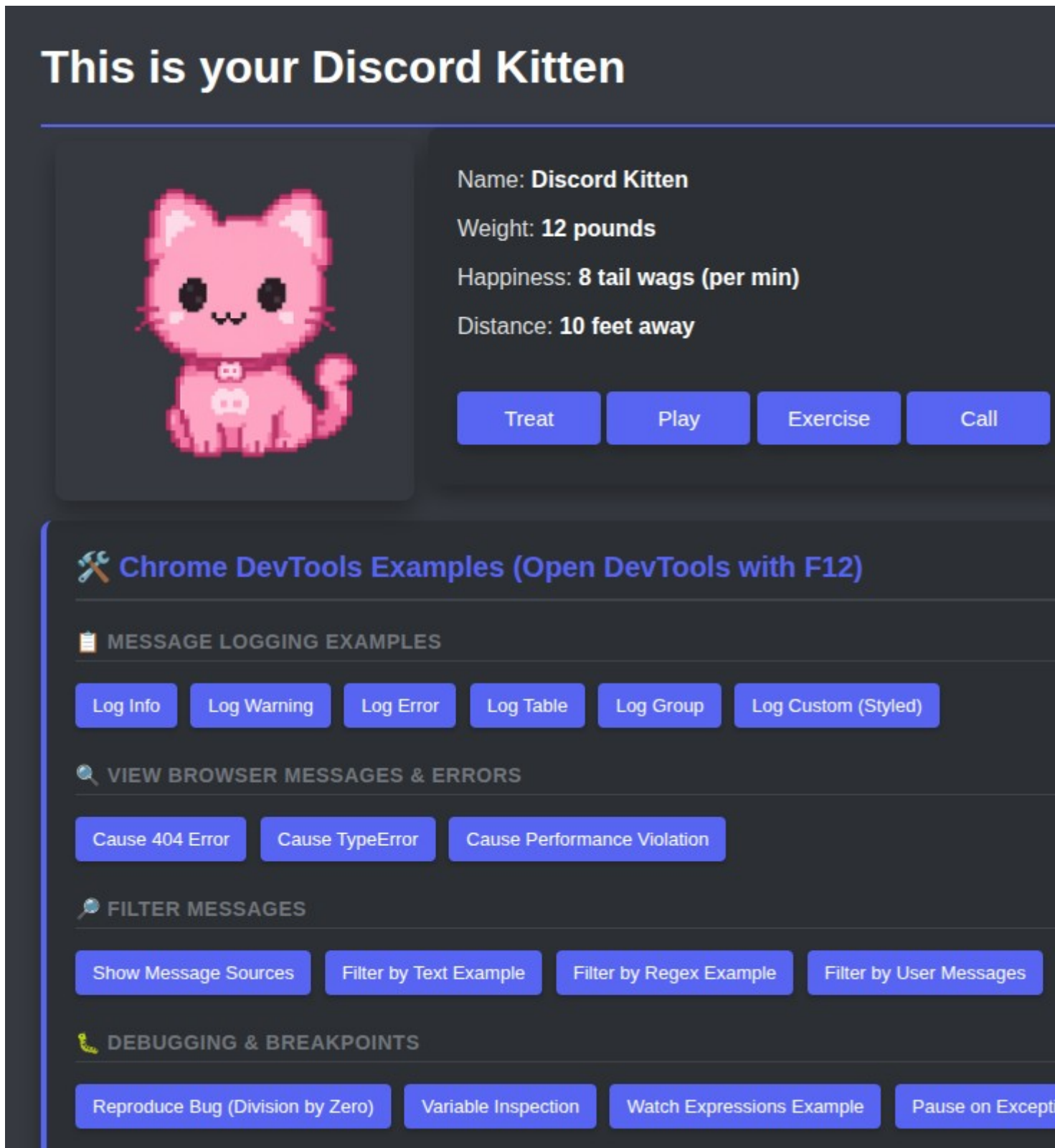
Search for messages containing specific text.

Step 1: Click "Filter by Text Example" button

This is the companion to the watch-expressions demo in the code.

Figure 15. App Overview Reference

The final capture closes the loop with the app itself, showing the DevTools demo controls that map to the guide sections throughout the report.



This is the page users interact with while collecting the live DevTools screenshots.

Summary

The project copy now serves as a practical DevTools exercise page and the PDF now includes the complete 15-figure screenshot sequence required by the assignment. It gives a clear path for reproducing a bug, inspecting execution state in Sources, and then applying the fix with the corrected division logic in the app.

Generated from the workspace copy of the Project 2 assignment for submission to the HW10 Chrome DevTools task.